Evaluación del módulo 6

Consigna del trabajo 👩🏻‍💻

Evaluación del módulo

Proyecto *Alke Wallet*

Situación inicial 📍

Formando parte de un grupo de desarrolladores, has sido encomendado para crear una billetera digital denominada "Wallet". Esta herramienta posibilitará a los clientes rastrear sus gastos y llevar a cabo transacciones virtuales.

En etapas previas, se ha construido la estructura básica de la aplicación, que abarca todas las interfaces necesarias y el flujo de información entre la visualización y la lógica de control. En esta etapa, el objetivo radica en culminar el proceso de desarrollo de la aplicación, incorporando la funcionalidad para establecer conexión con una API REST externa. Esto nos permitirá conferir funcionalidad al aplicativo, habilitando la ejecución de transacciones y la visualización del historial de operaciones ya efectuadas. De esta forma, la aplicación se volverá completamente operativa.

Nuestro objetivo 📋

El primer objetivo es llevar a cabo la funcionalidad principal de la aplicación, que es permitir a los usuarios efectuar transacciones virtuales. Esto implica implementar la lógica de comunicación con la API REST utilizando Retrofit.

Otro objetivo importante es implementar la base de datos local utilizando Room. Esto permitirá almacenar datos en el dispositivo del usuario para garantizar un acceso rápido y sin conexión a información relevante, como el historial de transacciones y la información del perfil del usuario.

Es crucial integrar todas las librerías y componentes de manera coherente en la arquitectura MVVM. Esto implica separar claramente la lógica de presentación de los datos y la lógica de negocio. La View se encargará de la interfaz de usuario, el ViewModel será responsable de coordinar la comunicación con Retrofit para acceder a la API REST, con Room para acceder a la base de datos local y con la View para mostrar los datos al usuario, mientras que el Model contendrá los datos.

La implementación debe garantizar una experiencia de usuario fluida y amigable. Esto incluye una navegación intuitiva entre las pantallas de la aplicación, así

como la presentación eficiente de información y la carga de imágenes sin inconvenientes.

Requerimientos 🤝

En primer lugar, es esencial brindar a los usuarios la capacidad de registrarse e iniciar sesión de manera segura. Esto implica la implementación de mecanismos de autenticación utilizando técnicas adecuadas, como tokens de seguridad y gestión de sesiones, para asegurar el acceso a la aplicación solo por usuarios autorizados.

La visualización de las transacciones realizadas y el seguimiento de los gastos son funcionalidades cruciales. Para lograrlo, se utilizará la librería Retrofit, que permitirá establecer una comunicación fluida con una API REST externa. Esta comunicación facilitará la obtención de los datos necesarios, como detalles de transacciones, para mostrarlos en la interfaz de usuario.

La posibilidad de realizar transacciones virtuales requiere una integración sin problemas con la API REST, lo cual será gestionado por Retrofit. Los datos necesarios para realizar transacciones serán enviados a través de solicitudes HTTP, asegurando que los movimientos se realicen de manera exitosa y segura.

Para permitir el acceso sin conexión y una respuesta rápida, la aplicación deberá almacenar los datos en una base de datos local. Para este propósito, se utilizará la librería Room. La implementación de Room permitirá guardar los detalles del perfil y las transacciones en el dispositivo del usuario, facilitando el acceso a la información incluso en ausencia de conectividad.

La carga y presentación de imágenes, como la imagen del perfil del usuario, se gestionará mediante la librería Picasso. Esto garantizará que las imágenes se carguen de manera eficiente y se muestren correctamente en la interfaz de usuario.

En resumen, la integración de librerías como Retrofit, Room y Picasso, junto con la adopción del patrón MVVM, garantizará que la aplicación "Wallet" cumpla con la funcionalidad esperada y ofrezca una experiencia fluida y segura a los usuarios.

Los requerimientos funcionales específicos podrían enumerarse en:

1. **Registro e inicio de sesión**: La aplicación debe permitir a los usuarios registrarse y luego iniciar sesión con sus credenciales.

2. **Historial de transacciones**: La aplicación debe mostrar un historial de transacciones, detallando la fecha, monto y descripción de cada una.

3. **Realización de transacciones virtuales**: Los usuarios deben poder efectuar transacciones virtuales, indicando el monto y una descripción opcional.

4. **Almacenamiento local**: La aplicación debe almacenar en una base de datos local el historial de transacciones y los detalles del perfil del usuario para un acceso sin conexión.

5. **Carga de imágenes**: La imágen del perfil del usuario debe cargarse y mostrarse adecuadamente en la interfaz.

Estos requerimientos funcionales guiarán la implementación técnica de la aplicación:

- **Librería Retrofit**: Implementar Retrofit para establecer comunicación con la API REST y realizar solicitudes HTTP para obtener y enviar datos.
- **Librería Room**: Utilizar Room para crear una base de datos local que almacene el historial de transacciones y los detalles del perfil del usuario.
- **Patrón de Arquitectura MVVM**: Diseñar la aplicación siguiendo el patrón de arquitectura MVVM, separando la lógica de presentación, la lógica de negocio y los datos.
- **Integración con Picasso**: Utilizar la librería Picasso para cargar y mostrar imágenes.
- **Gestión de sesiones y autenticación**: Implementar mecanismos de gestión de sesiones y autenticación segura para proteger los datos de los usuarios.
- **Manejo de errores**: Implementar manejo de errores adecuado al comunicarse con la API y al realizar operaciones locales, proporcionando mensajes claros al usuario en caso de fallos.
- **Seguridad de datos**: Garantizar la seguridad de los datos del usuario tanto en tránsito (mediante HTTPS) como en reposo (en la base de datos local).
- **Pruebas unitarias y de integración**: Realizar pruebas exhaustivas, incluyendo pruebas unitarias para las distintas capas de la aplicación y pruebas de integración para asegurar el correcto funcionamiento.

¿Qué vamos a validar? 🔍

Se validará que todas las funcionalidades establecidas en los requerimientos funcionales se implementen de manera correcta y completa. Esto incluye el registro, inicio de sesión, visualización de operaciones, realización de transacciones virtuales y almacenamiento local.

Se comprobará que las librerías utilizadas, como Retrofit, Room y Picasso, se integren de manera efectiva en la aplicación y cumplan con su propósito. Se evaluará si las solicitudes a la API REST se realizan correctamente, si la base de datos local funciona adecuadamente y si las imágenes se cargan sin bloquear la fluidez de la aplicación

Se esperan pruebas unitarias para garantizar que todas las funcionalidades respondan de manera correcta y sin errores.

Se evaluará el rendimiento de la aplicación, especialmente en lo que respecta a la velocidad de carga de datos y la respuesta a acciones del usuario, para garantizar una experiencia fluida.

Se verificará que los mecanismos de seguridad y autenticación estén implementados de manera adecuada, protegiendo los datos y la privacidad de los usuarios.

Se comprobará que la aplicación maneje de manera adecuada los posibles errores, como conexiones fallidas a la API, errores de autenticación o problemas en la base de datos local. Los mensajes de error deberán ser claros y comprensibles para los usuarios.

En resumen, se validarán varios aspectos para asegurarse de que la aplicación cumple con los requisitos y ofrece una experiencia de usuario sólida y confiable.

Referencias 🦺

La documentación oficial proporcionada por la entidad que disponibiliza la API REST servirá como guía esencial durante la fase de desarrollo del proyecto.

Es importante utilizar la documentación oficial para asegurarte de estar trabajando con información actualizada y precisa directamente proporcionada por los proveedores de la API. Esta referencia te ayudará a entender cómo interactuar con la API de manera efectiva y a utilizar los datos correctamente en tu aplicación.

Recursos 🎁

- API REST externa

Entregables ✅

Entregables Puntuales 📬

1. Arquitectura y Diseño MVVM:

   - Descripción: Entregar el diseño de la aplicación utilizando el patrón de arquitectura MVVM (Modelo-Vista-VistaModelo).
   - Requerimiento: Asegúrate de que la aplicación esté dividida correctamente en las tres capas principales:
     - Modelo (Model): Debe contener la lógica de los datos, incluyendo las entidades de la base de datos y las clases para acceder a la API.
     - Vista (View): La interfaz de usuario debe estar desacoplada de la lógica de negocio. Aquí se utilizarán fragmentos o actividades que solo interactúan con los datos a través del ViewModel.
     - VistaModelo (ViewModel): Debe gestionar la comunicación entre la vista y el modelo. Este componente debe manejar la lógica de la aplicación como las interacciones con Retrofit (para la API) y Room (para la base de datos local), y actualizar la vista con los datos.

2. Implementación de Room para almacenamiento local:

   - Descripción: Implementar la base de datos local utilizando Room para almacenar información como el historial de transacciones y el perfil del usuario.
   - Requerimiento:
     - Crear las entidades y las tablas necesarias para almacenar los datos locales.
     - Implementar la DAO (Data Access Object) para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre la base de datos.
     - Asegúrate de que la base de datos esté correctamente integrada en el ViewModel, de manera que los datos estén disponibles sin conexión.

3. Integración de Retrofit para la comunicación con la API REST:

- Descripción: Establecer la comunicación con la API REST utilizando Retrofit para realizar solicitudes HTTP y obtener datos.
- Requerimiento:
    - Implementar las interfaces de Retrofit para hacer las solicitudes GET y POST a la API REST externa.
    - Gestionar las respuestas de la API en el ViewModel y asegurar que los datos se actualicen en la vista.
    - Asegúrate de que las interacciones con Retrofit sigan buenas prácticas de manejo de errores y gestionen los posibles fallos de red de manera adecuada.

4. Implementación de la pantalla de inicio de sesión y registro:

- Descripción: Desarrollar las pantallas de inicio de sesión y registro de usuario, con validaciones y gestión de sesiones.
- Requerimiento:
    - Crear las pantallas para el registro de usuario, incluyendo campos para ingresar información de autenticación (nombre de usuario, correo, contraseña).
    - Implementar la funcionalidad de inicio de sesión, que debe autenticarse con un servicio API (aunque la parte de autenticación completa puede ser un alcance futuro, el diseño de la pantalla debe estar preparado).
    - Asegúrate de que la información se maneje correctamente (con tokens de autenticación o mecanismos similares) y que los datos sensibles estén gestionados de manera segura.

5. Manejo de errores:

- Descripción: Implementar un manejo de errores adecuado, tanto para las interacciones con Retrofit (API REST) como para las operaciones con Room (base de datos local).
- Requerimiento:
    - Asegúrate de que la aplicación maneje correctamente los posibles errores, como la falta de conectividad, problemas al acceder a la base de datos o fallos de autenticación.
    - Los mensajes de error deben ser claros y comprensibles para el usuario, evitando mostrar errores técnicos.
    - Además, implementar validaciones de datos en el registro e inicio de sesión para asegurar que los usuarios ingresen información válida.

6. Pruebas unitarias:

- Descripción: Es necesario implementar pruebas unitarias para las partes fundamentales del sistema.
- Requerimiento:
  - Crear pruebas unitarias para el ViewModel y el acceso a la base de datos con Room, asegurándote de que las interacciones entre el modelo y la base de datos funcionen correctamente.
  - Incluir pruebas de integración para garantizar que Retrofit realice las solicitudes correctamente y maneje las respuestas adecuadamente.

7. Documentación Técnica:

- Descripción: Documentar el proceso de implementación y el diseño de la arquitectura de la aplicación.
- Requerimiento:
  - Incluir una sección detallada sobre cómo está estructurado el código, las principales decisiones de diseño, cómo se gestiona la base de datos con Room y cómo se integra Retrofit para la API.
  - Proporcionar una descripción de las principales clases y sus responsabilidades dentro del patrón MVVM.

Entregables Esperados (Resumen):

- Arquitectura MVVM:
  - Implementación y justificación de la estructura de la aplicación basada en MVVM.
- Base de Datos Local (Room):
  - Implementación de Room para almacenar transacciones y perfil de usuario.
- Comunicación con API REST (Retrofit):
  - Configuración de Retrofit para hacer peticiones a la API.
- Pantallas de Registro e Inicio de Sesión:
  - Diseño de pantallas con validaciones para el registro e inicio de sesión.
- Manejo de Errores:
  - Gestión adecuada de errores en las interacciones con Retrofit y Room.
- Pruebas Unitarias:

    - Pruebas unitarias básicas de ViewModel y acceso a la base de datos.
- Documentación Técnica:
    - Documento con la descripción detallada de la arquitectura, decisiones de diseño y pruebas.

Portafolio 💼

No olvides subir tu trabajo al repositorio de GitHub y dar los accesos necesarios a quien corresponda para visualizar tu proyecto.